APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

# KTU CST414 — DEEP LEARNING

# MODULE 3 HANDWRITTEN LECTURE NOTES

**B.Tech Computer Science and Engineering · Scheme 2019**

| | |
|---|---|
| Course Code: CST414 | Credits: 3 |
| Course Name: Deep Learning | Lecture Hours: 36 |
| Faculty In-Charge: Jane Rose | Department: Computer Science & Engineering |
| Design: High-Contrast Printable Notebook | Font: Authentic Chalk & Kalam Ink |

*Prepared for students and faculty for classroom lectures, print study, and university examination preparation.*

## COURSE LECTURE INDEX & MAPPING

| Day | Topic ID | Lecture Title | Module | CO | Date |
|---|---|---|---|---|---|
| #17 | 3.1 | Convolutional Neural Networks: The Convo... | M3 | CO3 | 06-Oct-2025 |
| #18 | 3.2 | Motivation for CNNs and Pooling Operatio... | M3 | CO3 | 08-Oct-2025 |
| #19 | 3.3 | Convolution and Pooling as an Infinitely... | M3 | CO3 | 10-Oct-2025 |
| #20 | 3.4 | Variants of the Convolution Function | M3 | CO3 | 13-Oct-2025 |
| #21 | 3.5 | Structured Outputs and Diverse Data Type... | M3 | CO3 | 15-Oct-2025 |
| #22 | 3.6 | Efficient Convolution Algorithms | M3 | CO3 | 17-Oct-2025 |

# LECTURE #17 · MODULE 3 (Topic 3.1)

Date: 06-Oct-2025
Period: Period 2

## Title: Convolutional Neural Networks: The Convolution Operation

CO: CO3 Bloom's: Understand / Apply

Prerequisites: Matrix algebra, 2D grids, Multi-Layer Perceptrons, Day 8 concepts
Key Terminology: Convolution, Cross-Correlation, Filter Kernel, Feature Map, Kernel Flipping, Receptive Field, Discrete 2D Convolution

## Convolution Operation

Continuous 1D Convolution: s(t) = (x * w)(t) = integral_{-infinity}^{infinity} x(a) * w(t - a) da Discrete 2D Mathematical Convolution (with Kernel Flip): S(i, j) = (I * K)(i, j) = sum_m sum_n I(i - m, j - n) * K(m, n) Discrete 2D Cross-Correlation (Deep Learning Convention): S(i, j) = (I * K)(i, j) = sum_m sum_n I(i + m, j + n) * K(m, n) In deep learning libraries (PyTorch, TensorFlow), cross-correlation is implemented without flipping the kernel because kernel weights are learned directly from data via backpropagation, making learning flipped weights mathematically equivalent to learning unflipped weights.

## Feature Map And Filters

A filter kernel is a small matrix (e.g. 3x3, 5x5) whose elements act as feature detectors: - Horizontal Sobel Filter: K = [[-1, -2, -1], [0, 0, 0], [1, 2, 1]] - Vertical Sobel Filter: K = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]] Sliding this across an image generates a 2D scalar activation map highlighting local directional features.

## Exam Important Points

- Continuous vs discrete 2D convolution formulas.
- Why deep learning libraries implement cross-correlation rather than true flipped convolution.
- Manual calculation of 2D convolution on a 4x4 matrix with a 3x3 kernel.
- Hand-crafted kernels: Sobel horizontal and vertical edge detectors.

## Worked Numerical Problems (Hand-Derived Step-by-Step)

### Problem 1 (Basic): Manual 2D Discrete Convolution Calculation

Statement: Given a 4x4 input image I = [[1, 2, 0, 1], [0, 1, 3, 2], [2, 0, 1, 0], [1, 1, 0, 2]] and a 3x3 kernel K = [[1, 0, -1], [1, 0, -1], [1, 0, -1]]. Calculate the valid 2D cross-correlation output feature map S.

Given: Input 4x4, Kernel 3x3, stride=1, no padding.

**Governing Formula: Output shape = (4 - 3 + 1) x (4 - 3 + 1) = 2x2. S(i, j) = sum(I_patch * K)**

**Step-by-Step Calculation:**

Step 1: Output dimension is 2x2: S(0,0), S(0,1), S(1,0), S(1,1).
Step 2: S(0,0) = (1*1 + 2*0 + 0*-1) + (0*1 + 1*0 + 3*-1) + (2*1 + 0*0 + 1*-1) = 1 - 3 + 1 = -1.
Step 3: S(0,1) = (2*1 + 0*0 + 1*-1) + (1*1 + 3*0 + 2*-1) + (0*1 + 1*0 + 0*-1) = 1 - 1 + 0 = 0.
Step 4: S(1,0) = (0*1 + 1*0 + 3*-1) + (2*1 + 0*0 + 1*-1) + (1*1 + 1*0 + 0*-1) = -3 + 1 + 1 = -1.
Step 5: S(1,1) = (1*1 + 3*0 + 2*-1) + (0*1 + 1*0 + 0*-1) + (1*1 + 0*0 + 2*-1) = -1 + 0 - 1 = -2.

**[ FINAL EVALUATED ANSWER: Feature Map S = [[-1, 0], [-1, -2]] ]**

Pedagogical Note: The 3x3 vertical difference kernel extracts spatial vertical edge gradients across the 4x4 grid.

### Problem 2 (Advanced (KTU Exam Level)): Mathematical Equivalence of Cross-Correlation and Convolution

Statement: Let continuous 1D convolution be (f * g)(t) = integral f(tau) * g(t - tau) d tau and cross-correlation be (f star g)(t) = integral f(tau) * g(t + tau) d tau. Prove that (f star g)(t) = (f * g_minus)(t) where g_minus(t) = g(-t), and explain why deep learning frameworks can optimize cross-correlation filters directly without explicit flipping.

Given: Definitions of convolution, cross-correlation, and flipped kernel g_minus(t) = g(-t).

**Governing Formula: (f * g_minus)(t) = integral f(tau) * g_minus(t - tau) d tau**

#### Step-by-Step Calculation:

Step 1: Write convolution: (f * g_minus)(t) = integral f(tau) * g_minus(t - tau) d tau.

Step 2: Substitute g_minus(u) = g(-u): g_minus(t - tau) = g(-(t - tau)) = g(tau - t).

Step 3: In 2D arrays: K_flipped[i, j] = K[-i, -j]. Thus cross-correlation with K equals convolution with K_flipped.

Step 4: In deep learning, weights in K are free parameters learned end-to-end to minimize loss; the optimizer learns the optimal orientation directly, eliminating the need to reverse arrays at runtime.

**[ FINAL EVALUATED ANSWER: Cross-correlation with kernel K is mathematically identical to true convolution with flipped kernel K_minus. ]**

Pedagogical Note: End-to-end learning renders the mathematical difference between convolution and cross-correlation a mere indexing convention.

### Real-World Industry Case (Semiconductor Manufacturing): Silicon Wafer Defect Inspection

Pipeline Workflow: High-res microscope scans wafer -> 2D convolution applies learned micro-crack filters -> Edge-enhanced feature maps identify hairline sub-micron etch defects.

**Industrial Impact & Benefit: Prevents defective silicon chips from reaching automotive and aerospace electronic engine control units.**

## Key Oral Viva Voce Checkpoints

**Q1: What is the difference between mathematical convolution and cross-correlation?**

Ans: Mathematical convolution flips the kernel by 180 degrees before computing the inner product; cross-correlation slides the kernel without flipping.

**Q2: Why do deep learning frameworks implement cross-correlation while calling it convolution?**

Ans: Because kernel weights are learned from data; learning flipped weights is mathematically equivalent to learning unflipped weights, so flipping is an unnecessary computational overhead.

**Q3: What is a feature map?**

Ans: The 2D output array produced by sliding a convolutional filter across an input image and computing local inner products.

**Q4: What determines output spatial dimensions of a 2D convolution without padding?**

Ans: Output Height = H_in - K_h + 1, Output Width = W_in - K_w + 1.

**Q5: What does a 3x3 Sobel kernel detect?**

Ans: Spatial image intensity gradients (directional edges) along horizontal or vertical orientations.

# LECTURE #18 · MODULE 3 (Topic 3.2)

Date: 08-Oct-2025
Period: Period 4

## Title: Motivation for CNNs and Pooling Operations

CO: CO3 Bloom's: Understand / Apply

Prerequisites: Convolution Operation, Multi-Layer Perceptrons, Day 17 concepts
Key Terminology: Sparse Interactions, Parameter Sharing, Equivariance to Translation, Max Pooling, Average Pooling, Translation Invariance, Downsampling

## Three Motivations For Cnns

1. Sparse Interactions: Kernel size k << input size m. Receptive fields are local, reducing storage from O(m*n) to O(k*n) and slashing FLOPs. 2. Parameter Sharing: Reusing kernel weights across all positions slashes parameter counts and prevents overfitting. 3. Equivariant Representations: f(g(x)) = g(f(x)). Shifting the input image translates feature activations identically.

## Pooling Operations

Pooling summarizes nearby outputs: - Max Pooling: S(i, j) = max_{(p,q) in R} I(i+p, j+q) (captures dominant activations). - Average Pooling: S(i, j) = (1/|R|) sum_{(p,q) in R} I(i+p, j+q) (smooth background summary). Pooling provides approximate invariance to small translations of the input.

## Exam Important Points

- Sparse Interactions, Parameter Sharing, and Translation Equivariance.
- Translation Equivariance (convolution) vs Translation Invariance (pooling).
- Max Pooling vs Average Pooling manual calculation on a 4x4 matrix.
- How pooling reduces spatial dimensions and controls overfitting.

## Worked Numerical Problems (Hand-Derived Step-by-Step)

### Problem 1 (Basic): Max and Average Pooling Calculation

Statement: Given 4x4 matrix M = [[12, 20, 30, 0], [8, 16, 2, 14], [4, 6, 8, 10], [18, 22, 12, 14]], perform 2x2 pooling with stride 2: (a) Compute Max Pooling, and (b) Compute Average Pooling.

Given: 4x4 matrix, pool_size=2, stride=2.
**Governing Formula: Max = max(patch); Avg = mean(patch)**

**Step-by-Step Calculation:**

Q1 [[12, 20], [8, 16]]: Max = 20, Avg = 56/4 = 14.0.
Q2 [[30, 0], [2, 14]]: Max = 30, Avg = 46/4 = 11.5.
Q3 [[4, 6], [18, 22]]: Max = 22, Avg = 50/4 = 12.5.
Q4 [[8, 10], [12, 14]]: Max = 14, Avg = 44/4 = 11.0.

**[ FINAL EVALUATED ANSWER: Max Pool = [[20, 30], [22, 14]], Avg Pool = [[14.0, 11.5], [12.5, 11.0]] ]**
Pedagogical Note: Max pooling captures peak signals while average pooling extracts mean background response.

### Problem 2 (Advanced (KTU Exam Level)): Sparse Interactions vs Dense MLP Parameter Reduction

Statement: An image with resolution 1000x1000 is processed by a hidden layer of 1000x1000 units. (a) Calculate weights in fully connected MLP, (b) Calculate weights in CNN with a 5x5 filter and 1 feature map, (c) Compute parameter reduction ratio, and (d) Calculate parameters with 16 filters.

Given: Input 10^6, Hidden 10^6, Kernel 5x5.

**Governing Formula: MLP = m * n; CNN = k_h * k_w * filters**

#### Step-by-Step Calculation:

Step 1: MLP weights = 10^6 * 10^6 = 10^12 = 1 Trillion parameters.
Step 2: CNN weights (1 filter) = 5 * 5 = 25 parameters.
Step 3: Reduction ratio = 10^12 / 25 = 40,000,000,000x.
Step 4: CNN weights (16 filters) = 16 * 25 = 400 parameters.

**[ FINAL EVALUATED ANSWER: MLP = 1 Trillion weights. CNN (1 filter) = 25 weights (40B x reduction). CNN (16 filters) = 400 weights. ]**

Pedagogical Note: CNNs make high-resolution vision computationally tractable through parameter sharing.

### Real-World Industry Case (Biometrics & Airport Security): Biometric Facial Recognition Gates

Pipeline Workflow: Passenger walks past camera -> Head position varies by centimeters -> Max pooling provides translation invariance -> Produces stable face embeddings.

**Industrial Impact & Benefit: Enables seamless passenger boarding without requiring rigid motionless posture alignment.**

## Key Oral Viva Voce Checkpoints

**Q1: What are the three core motivations of CNNs?**
Ans: Sparse Interactions, Parameter Sharing, and Equivariant Representations.

**Q2: What is translation equivariance?**
Ans: Shifting input causes output feature map to shift by the identical amount: f(g(x)) = g(f(x)).

**Q3: Does max pooling contain trainable weights?**
Ans: No, it is a fixed deterministic non-linear mathematical operation with zero learnable parameters.

**Q4: What is Global Average Pooling?**
Ans: Averaging all spatial values across an entire 2D channel to produce a single scalar per feature map.

**Q5: How does pooling reduce compute in deeper layers?**
Ans: Downsampling height and width by 2x reduces spatial locations for subsequent layers by 75%.

# LECTURE #19 · MODULE 3 (Topic 3.3)

Date: 10-Oct-2025
Period: Period 1

## Title: Convolution and Pooling as an Infinitely Strong Prior

CO: CO3 Bloom's: Understand / Analyze

*Prerequisites: Bayesian Probability (priors, posteriors), Convolution and Pooling mechanics, Day 18 concepts*
*Key Terminology: Bayesian Prior, Infinitely Strong Prior, Inductive Bias, Parameter Constraint, Permutation Vulnerability, Spatial Locality Assumption*

## Bayesian Prior Perspective

*In Bayesian statistics, a prior probability distribution p(theta) reflects our beliefs about which parameter configurations are plausible before observing any training data. A prior is 'weak' or 'soft' if it assigns positive probability to all parameter values, merely preferring some over others (e.g. Gaussian prior in L2 regularization with high variance). A prior is 'infinitely strong' if it assigns zero probability to certain parameter configurations, making it mathematically impossible for the learning algorithm to ever choose those weights regardless of how much training data is collected.*

## Convolution And Pooling As Infinitely Strong Prior

*A Convolutional layer can be viewed as a fully connected layer with an infinitely strong prior specifying that: 1. All weights connecting non-spatially adjacent pixels must be identically ZERO: p(W_ij != 0) = 0 for |i - j| > kernel_radius. 2. Weights for spatially shifted connections must be EXACTLY IDENTICAL (parameter sharing): p(W_ij != W_{i+1, j+1}) = 0. Similarly, a Pooling layer is an infinitely strong prior specifying that each unit must be invariant to small local translations. Benefits and Hazards: - Benefit: If the prior assumption is TRUE (e.g. in natural images where pixels have local stationary correlations), the CNN learns with massive sample efficiency, avoiding overfitting. - Hazard: If the prior assumption is FALSE (e.g. tabular data where column ordering is arbitrary, or permuted images), an infinitely strong prior prevents the network from learning the true relationship, leading to high underfitting bias.*

## Permutation Experiment

*If we randomly permute all pixels of MNIST images with a fixed permutation matrix P: - A fully connected MLP achieves the EXACT same accuracy on permuted MNIST as on unpermuted MNIST (MLPs have a weak, isotropic prior). - A CNN's performance collapses dramatically on permuted MNIST because the infinitely strong spatial locality prior is violently violated!*

## Exam Important Points

- Definition of weak vs infinitely strong Bayesian priors.
- How convolution enforces an infinitely strong prior: zero weights outside kernel + identical shifted weights.
- Explanation of the Permuted MNIST experiment comparing MLPs and CNNs.
- When infinitely strong priors lead to catastrophic underfitting.

## Worked Numerical Problems (Hand-Derived Step-by-Step)

### Problem 1 (Basic): Weight Matrix Density under Infinitely Strong Prior

Statement: Consider a 1D input of length N = 100 mapped to an output of length N = 100. (a) In a dense MLP, calculate the total number of weight connections in the 100x100 matrix, (b) In a convolutional layer with kernel size k = 5 and circular padding, calculate the number of non-zero entries in the matrix, and (c) Determine the percentage of weights forced to be ZERO by the infinitely strong spatial locality prior.

Given: N = 100, k = 5.

**Governing Formula: MLP entries = N^2; Conv non-zero entries = N * k; Sparsity % = (1 - Non2ero / Total) * 100**

#### Step-by-Step Calculation:

Step 1: Dense MLP total weight matrix entries = 100 * 100 = 10,000 entries.

Step 2: Convolutional kernel k = 5 means each row has exactly 5 non-zero entries. Total non-zero entries across 100 rows = 100 * 5 = 500 entries.

Step 3: Number of zero-constrained entries = 10,000 - 500 = 9,500 entries.

Step 4: Percentage of weights forced to zero = (9,500 / 10,000) * 100 = 95.0%.

**[ FINAL EVALUATED ANSWER: Non-zero entries = 500; Zero-constrained entries = 9,500 (95.0% forced to zero). ]**

Pedagogical Note: The infinitely strong prior zeroes out 95% of all conceivable linear connections and constrains the remaining 500 entries to share only 5 distinct parameter values.

### Problem 2 (Advanced (KTU Exam Level)): Bayesian Gaussian Prior vs Infinitely Strong Prior Variance Limit

Statement: In Bayesian linear regression, weights have a zero-mean Gaussian prior p(w) = N(0, sigma_prior^2 * I). (a) Write the MAP regularized objective function, (b) What happens to the prior as sigma_prior^2 -> infinity? (c) What happens to the prior as sigma_prior^2 -> 0? and (d) Explain how an infinitely strong prior relates to setting prior variance to zero for non-local weights.

Given: p(w) = N(0, sigma_prior^2 * I), Loss = L_data - ln p(w).

**Governing Formula: J_MAP(w) = J_data(w) + (1 / (2 * sigma_prior^2)) * ||w||_2^2**

#### Step-by-Step Calculation:

Step 1: Write negative log posterior: J_MAP(w) = J_data(w) + (1 / (2 * sigma_prior^2)) * sum_i w_i^2 + const. This is exact L2 regularization with penalty alpha = 1 / sigma_prior^2.

Step 2: Limit as sigma_prior^2 -> infinity: alpha -> 0. The prior becomes an improper uniform distribution, exerting zero constraint on weights (unregularized maximum likelihood).

Step 3: Limit as sigma_prior^2 -> 0: alpha -> infinity. The penalty for any non-zero weight w_i != 0 approaches infinity, forcing w_i to be identically zero with probability 1.

Step 4: Connection to CNNs: A CNN sets sigma_prior^2 = 0 for all weights connecting pixels outside the local kernel window, creating a Dirac delta prior delta(w_ij) that permanently forbids non-local interactions.

**[ FINAL EVALUATED ANSWER: sigma_prior^2 -> 0 yields an infinitely strong prior forcing non-local weights to zero. ]**

Pedagogical Note: Architectural constraints in CNNs are equivalent to infinitely strong Bayesian priors with zero variance on non-local weights.

### Real-World Industry Case (Bioinformatics & Computational Genomics): Genomic Variant Pathogenicity Modeling

Pipeline Workflow: DNA sequences have local spatial motif structure (transcription factor binding sites) -> CNNs with local 1D convolution priors detect regulatory motifs -> Tabular patient clinical data (age, lab blood values) lacks spatial ordering -> Processed via gradient boosted trees or MLPs to avoid harmful spatial priors.

**Industrial Impact & Benefit: Applying the correct inductive bias prevents misinterpreting unstructured patient records while capturing sequential genomic motifs.**

## Key Oral Viva Voce Checkpoints

**Q1: What is an infinitely strong prior in Bayesian terms?**

*Ans: A prior distribution that assigns zero probability to certain parameter values, completely forbidding the model from adopting those parameters regardless of the training data.*

**Q2: How does a convolutional layer represent an infinitely strong prior?**

*Ans: By forcing all weights outside the local kernel window to be identically zero and forcing weights at different spatial positions to be identical (parameter sharing).*

**Q3: What happens if you train a CNN on images with randomly permuted pixels?**

*Ans: Performance degrades severely because the spatial locality and translation invariance priors are violated.*

**Q4: Why does an MLP perform identically on original and permuted MNIST?**

*Ans: Because an MLP is permutation invariant with respect to its input features; it possesses no spatial locality prior.*

**Q5: When is an infinitely strong prior beneficial?**

*Ans: When the underlying physical process satisfies the prior assumptions (e.g. 2D natural images possessing translation invariance and local correlations).*

# LECTURE #20 · MODULE 3 (Topic 3.4)

Date: 13-Oct-2025
Period: Period 2

## Title: Variants of the Convolution Function

CO: CO3 Bloom's: Understand / Apply

*Prerequisites: Convolution Operation, Day 17 concepts*
*Key Terminology: Strided Convolution, Valid Padding, Same Padding, Full Padding, Locally Connected Layers, Tiled Convolution, 1x1 Convolution, Dilated (Atrous) Convolution*

## Strided Convolution

*Standard convolution slides the kernel by 1 pixel at each step. A strided convolution slides the kernel by s pixels (s > 1): S(i, j) = sum_m sum_n I(i*s + m, j*s + n) * K(m, n) Effect: Downsamples the feature map by a factor of s without requiring a separate pooling layer. Modern architectures (ResNet, MobileNet) frequently replace max pooling with strided convolutions.*

## Zero Padding Types

*For input dimension W, kernel size K, stride s, and zero-padding p: Output Dimension: W_out = floor( (W - K + 2*p) / s ) + 1 1. Valid Padding (p = 0): No padding added. W_out = (W - K + 1). Feature maps shrink at every layer. 2. Same Padding (p = (K - 1) / 2 for odd K, s = 1): Zeros added such that output spatial dimensions exactly match input dimensions: W_out = W. 3. Full Padding (p = K - 1, s = 1): Zeros added such that every pixel is visited K times: W_out = W + K - 1.*

## Specialized Convolution Variants

*1. 1x1 Convolution (Lin et al., Network in Network, 2013): Acts as a cross-channel linear combination of features without altering spatial resolution. Crucial for channel dimensionality reduction (e.g. Inception bottlenecks, ResNet residual blocks). 2. Dilated (Atrous) Convolution (Yu & Koltun, 2015): Inserts d - 1 zeros (holes) between kernel elements. Effective kernel size: K_eff = K + (K - 1) * (d - 1). Exponentially expands receptive field without increasing parameter count or losing spatial resolution (widely used in WaveNet and DeepLab semantic segmentation). 3. Locally Connected Layers (Unshared Convolutions): Local receptive fields without parameter sharing. Used when features are not stationary across the grid (e.g. human face alignment in DeepFace where eyes are always near the top).*

## Exam Important Points

- *Master formula for output spatial dimension: floor((W - K + 2p)/s) + 1.*
- *Definitions and diagrams for Valid, Same, and Full padding.*
- *Purpose and parameter reduction of 1x1 convolutions in Inception/ResNet.*
- *Dilated convolution receptive field formula and application in semantic segmentation.*

## Worked Numerical Problems (Hand-Derived Step-by-Step)

### Problem 1 (Basic): Convolution Output Shape Calculation across Padding Modes

Statement: An input feature map has spatial resolution W_in = 32x32. A convolutional layer uses filter size K = 5x5 with stride s = 1. Calculate the output spatial dimensions for: (a) Valid padding (p = 0), (b) Same padding, and (c) Full padding.

Given: W_in = 32, K = 5, s = 1.

**Governing Formula: W_out = floor((W_in - K + 2*p)/s) + 1; Same: p = (K - 1)/2; Full: p = K - 1**

#### Step-by-Step Calculation:

Step 1: Valid Padding (p = 0): W_out = (32 - 5 + 2*0)/1 + 1 = 27 + 1 = 28x28.

Step 2: Same Padding: p = (5 - 1) / 2 = 2 pixels on each side. W_out = (32 - 5 + 2*2)/1 + 1 = (32 - 5 + 4) + 1 = 31 + 1 = 32x32.

Step 3: Full Padding: p = K - 1 = 5 - 1 = 4 pixels on each side. W_out = (32 - 5 + 2*4)/1 + 1 = (32 - 5 + 8) + 1 = 35 + 1 = 36x36.

**[ FINAL EVALUATED ANSWER: Valid = 28x28, Same = 32x32, Full = 36x36 ]**

Pedagogical Note: Valid padding shrinks the map by 4 pixels, Same padding preserves the 32x32 resolution, and Full padding expands it to 36x36.

### Problem 2 (Advanced (KTU Exam Level)): KTU Model Paper Q9: Stride Effect & 1x1 Bottleneck Parameter Savings

Statement: Part 1: What happens if the stride of a convolutional layer increases? Part 2: A convolutional layer maps 256 input channels to 256 output channels using 3x3 filters. An alternative Bottleneck design uses: (i) a 1x1 convolution reducing 256 channels to 64, (ii) a 3x3 convolution with 64 channels, and (iii) a 1x1 convolution expanding 64 channels back to 256. Calculate the total weight parameters for both designs and the percentage reduction.

Given: Direct: 256 -> 256 with 3x3. Bottleneck: 256 -> 64 (1x1), 64 -> 64 (3x3), 64 -> 256 (1x1).

**Governing Formula: Params = C_in * C_out * K_h * K_w**

#### Step-by-Step Calculation:

Part 1: Increasing stride s reduces output spatial resolution by approximately 1/s, reducing memory and computation for subsequent layers while increasing the effective receptive field stride.

Part 2 (Direct Design): Weights = 256 * 256 * (3 * 3) = 65,536 * 9 = 589,824 parameters.

Part 3 (Bottleneck Design): Layer 1 (1x1): 256 * 64 * (1 * 1) = 16,384. Layer 2 (3x3): 64 * 64 * (3 * 3) = 4,096 * 9 = 36,864. Layer 3 (1x1): 64 * 256 * (1 * 1) = 16,384. Total Bottleneck Parameters = 16,384 + 36,864 + 16,384 = 69,632 parameters.

Part 4: Compute savings: Reduction = (589,824 - 69,632) / 589,824 * 100 = (520,192 / 589,824) * 100 = 88.19%.

**[ FINAL EVALUATED ANSWER: Direct = 589,824 parameters. Bottleneck = 69,632 parameters (88.19% parameter reduction). ]**

Pedagogical Note: Using 1x1 bottleneck convolutions cuts parameters by over 88% while preserving representational capacity, which is the cornerstone of modern ResNet and Inception architectures.

### Real-World Industry Case (Autonomous Vehicles & Scene Parsing): DeepLab Autonomous Driving Semantic Segmentation

Pipeline Workflow: High-res 1080p vehicle camera feed -> Standard pooling loses fine boundary details of distant pedestrians -> Dilated (Atrous) Convolutions expand receptive fields to cover entire intersections without downsampling.

**Industrial Impact & Benefit: Enables pixel-accurate sidewalk, curb, and pedestrian segmentation without spatial detail loss.**

## Key Oral Viva Voce Checkpoints

**Q1: What is the mathematical formula for the output dimension of a 2D convolution?**

*Ans: Output = floor((Input - Kernel + 2*Padding) / Stride) + 1.*

**Q2: What is the difference between Valid padding and Same padding?**

*Ans: Valid padding adds zero zeros (output shrinks by K - 1); Same padding adds enough zeros such that the output spatial resolution matches the input resolution when stride=1.*

**Q3: What is a 1x1 convolution and what is its primary use?**

*Ans: A convolution with a 1x1 kernel that computes a linear combination across input channels at each pixel, used for channel dimensionality reduction (bottlenecks).*

**Q4: What is a Dilated (Atrous) Convolution?**

*Ans: A convolution where the filter kernel is expanded with spaces/zeros between weights, enlarging the receptive field without adding parameters.*

**Q5: What are Locally Connected Layers?**

*Ans: Layers with local receptive fields like convolution, but without parameter sharing; each spatial location learns its own unique set of weights.*

# LECTURE #21 · MODULE 3 (Topic 3.5)

Date: 15-Oct-2025
Period: Period 4

## Title: Structured Outputs and Diverse Data Types in CNNs

CO: CO3 Bloom's: Understand / Apply

*Prerequisites: Convolution Operations, Multi-channel Tensors, Day 20 concepts*
*Key Terminology: Structured Outputs, Multi-Channel Tensors, 1D Convolution, 3D Volumetric Convolution, Semantic Segmentation, Object Detection Bounding Box, Spatiotemporal Feature Learning*

## Data Type Taxonomy

*CNNs process multi-dimensional grid data across diverse domains: 1. 1D Sequential Grids (Channels x Time): - Audio waveforms (1D amplitude sampled at 16 kHz or 44.1 kHz). - Biomedical ECG / EEG sensor telemetry. - Text sentences (sequence of word embedding vectors). 2. 2D Spatial Grids (Height x Width x Channels): - Standard RGB color photographs (H x W x 3). - Audio spectrograms (Time x Frequency x 1). - Multispectral satellite imagery (e.g. 13 spectral bands in Sentinel-2). 3. 3D Volumetric & Spatiotemporal Grids (Height x Width x Depth/Time x Channels): - Medical imaging: 3D MRI and CT volumetric anatomical voxel scans. - Video clips: 2D spatial frames concatenated over time (H x W x T x 3). - Computational fluid dynamics and weather simulation grids.*

## Structured Outputs

*Rather than outputting a single categorical scalar y in {1, ..., K}, structured prediction tasks output high-dimensional structured arrays: 1. Semantic Segmentation (Dense Pixel Classification): - Output is a 2D label map S of shape (H x W x C) where every individual pixel is assigned a class probability distribution. - Architecture: Fully Convolutional Networks (FCN) and U-Net using encoder-decoder paths and transposed convolutions (skip connections). 2. Object Detection Bounding Boxes: - Output combines classification probabilities with regression coordinates [x_center, y_center, width, height] for multiple bounding boxes (e.g. YOLO, Faster R-CNN).*

## Exam Important Points

- *Dimensional structure of 1D, 2D, and 3D convolutional tensors.*
- *How 3D convolutions extract spatiotemporal features from video clips.*
- *Difference between image classification and dense semantic segmentation.*
- *U-Net architecture with skip connections for medical image segmentation.*

## Worked Numerical Problems (Hand-Derived Step-by-Step)

### Problem 1 (Basic): 3D Volumetric Convolution Output Dimension and Parameter Count

Statement: A medical MRI brain scan has volumetric resolution 128x128x64 voxels (Height x Width x Depth) with 1 input channel. A 3D convolutional layer applies 32 filters of size 3x3x3 with stride s = 1 and no padding (valid). (a) Calculate the output volumetric dimensions, and (b) Calculate the total number of trainable weight and bias parameters in the layer.

Given: Input: 128x128x64, C_in = 1, Filters = 32, K = 3x3x3, s = 1, p = 0.

**Governing Formula: Out = floor((Dim - K + 2*p)/s) + 1; Weights = N_filters * C_in * (K_h * K_w * K_d); Biases = N_filters**

#### Step-by-Step Calculation:

Step 1: Compute output dimensions: H_out = 128 - 3 + 1 = 126. W_out = 128 - 3 + 1 = 126. D_out = 64 - 3 + 1 = 62. Output volume = 126 x 126 x 62 with 32 channels.

Step 2: Compute weights: Weights = 32 * 1 * (3 * 3 * 3) = 32 * 27 = 864 weights.

Step 3: Compute biases = 32. Total parameters = 864 + 32 = 896 parameters.

**[ FINAL EVALUATED ANSWER: Output dimensions = 126 x 126 x 62 (32 channels); Total parameters = 896 ]**

Pedagogical Note: 3D convolutions slide across height, width, and depth simultaneously, capturing volumetric anatomical spatial relationships.

### Problem 2 (Advanced (KTU Exam Level)): Memory Footprint in Dense Semantic Segmentation vs Image Classification

Statement: An image with resolution 512x512x3 is processed for: (Case A) 1000-class Image Classification using Global Average Pooling, and (Case B) 20-class Semantic Segmentation producing a full-resolution dense prediction map. Both outputs use 32-bit floats. Calculate: (a) Output tensor dimensions for Case A and Case B, (b) Output memory consumption for a single image in bytes and kilobytes, and (c) Discuss the gradient backpropagation memory challenge in dense segmentation.

Given: Image: 512x512x3. Case A: 1000 classes. Case B: 512x512x20 classes. Float32 (4 bytes).

**Governing Formula: Memory = Total_Elements * 4 bytes**

#### Step-by-Step Calculation:

Step 1: Case A (Classification): Output shape = 1 x 1000. Elements = 1000. Memory = 1000 * 4 bytes = 4,000 bytes = 3.91 KB.

Step 2: Case B (Semantic Segmentation): Output shape = 512 x 512 x 20. Elements = 512 * 512 * 20 = 262,144 * 20 = 5,242,880 elements. Memory = 5,242,880 * 4 bytes = 20,971,520 bytes = 20.0 MB!

Step 3: Comparison ratio: Ratio = 20,971,520 / 4,000 = 5,242.88x more output memory.

Step 4: Backpropagation implications: Storing full-resolution intermediate feature activations for a batch of 16 segmentation maps requires gigabytes of GPU VRAM, requiring gradient checkpointing or smaller mini-batch sizes.

**[ FINAL EVALUATED ANSWER: Classification = 3.91 KB. Segmentation = 20.0 MB (5,243x larger memory footprint). ]**

Pedagogical Note: Dense structured prediction tasks require immense GPU VRAM to cache high-resolution multi-channel tensors during the backward pass.

### Real-World Industry Case (Medical Radiology & Neurosurgical Navigation): Brain Glioblastoma Tumor Segmentation in 3D MRI

Pipeline Workflow: Patient multi-parametric 3D MRI scans (T1, T1-contrast, T2, FLAIR) -> 3D U-Net processes 4-channel volumetric voxels -> Produces voxel-wise segmented 3D tumor boundary mesh.

**Industrial Impact & Benefit: Enables neurosurgeons to achieve millimeter-accurate tumor resection while avoiding critical motor cortex brain tissue.**

## Key Oral Viva Voce Checkpoints

**Q1: What is the difference between image classification and semantic segmentation?**

Ans: Classification assigns a single label to the entire image; semantic segmentation assigns a categorical class label to every individual pixel in the image.

**Q2: What is a 1D convolution and where is it applied?**

Ans: A convolution where the kernel slides across a single spatial or temporal dimension, used in audio waveforms, text sequences, and ECG signals.

**Q3: What is a 3D convolution and where is it applied?**

Ans: A convolution where a 3D kernel slides across height, width, and depth/time, used in 3D MRI/CT scans and video action recognition.

**Q4: What is a U-Net architecture?**

Ans: An encoder-decoder architecture with skip connections between corresponding contracting and expanding levels, widely used for biomedical image segmentation.

**Q5: Why are skip connections critical in semantic segmentation architectures?**

Ans: They bypass deep bottleneck downsampling, transferring high-resolution spatial boundary details directly to the decoder for precise pixel localization.

# LECTURE #22 · MODULE 3 (Topic 3.6)

Date: 17-Oct-2025
Period: Period 1

## Title: Efficient Convolution Algorithms

CO: CO3 Bloom's: Understand / Apply

Prerequisites: Convolution Operations, Matrix Multiplication, Complexity Theory, Day 21 concepts
Key Terminology: Efficient Convolutions, Depthwise Separable Convolution, Pointwise Convolution, Fast Fourier Transform (FFT), Convolution Theorem, Winograd Algorithm, MobileNet

## Computational Bottleneck

Standard 2D Convolution on an input of size D_F x D_F with M channels, producing N channels with kernel size D_K x D_K requires: Total Multiplications = D_F * D_F * M * N * D_K * D_K For a 56x56 feature map with M=256, N=512, and 3x3 kernel: 3.71 BILLION multiplications per forward pass! This is completely prohibitive on mobile edge hardware.

## Depthwise Separable Convolutions

MobileNet (Howard et al., 2017) factorizes standard convolution into two independent steps: 1. Depthwise Convolution: Applies a single spatial filter per input channel (M filters of size D_K x D_K x 1). Cost = D_F * D_F * M * D_K * D_K 2. Pointwise Convolution: A 1x1 convolution combining channels linearly across depth (N filters of size 1 x 1 x M). Cost = D_F * D_F * M * N Total Cost = D_F * D_F * M * (D_K^2 + N) Computation Reduction Ratio: Ratio = [ D_F^2 * M * (D_K^2 + N) ] / [ D_F^2 * M * N * D_K^2 ] = (D_K^2 + N) / (N * D_K^2) = (1 / N) + (1 / D_K^2) For standard 3x3 filters (D_K = 3) and large N (e.g. N = 512): Ratio approx 1 / 3^2 = 1 / 9 approx 0.111 (8 to 9 times faster, an 88-89% computational reduction with negligible accuracy loss!).

## Fft And Winograd Algorithms

1. Fast Fourier Transform (FFT) Convolution: By the Convolution Theorem: Convolution in the spatial domain equals point-wise multiplication in the frequency domain: I * K = IFFT( FFT(I) odot FFT(K) ) Complexity drops from O(N_pixels * K^2) to O(N_pixels * log N_pixels). Highly efficient for large kernels (K >= 7x7). 2. Winograd Minimal Filtering Algorithm (F(2x2, 3x3)): Lavin & Gray (2016) showed that computing 2x2 outputs with a 3x3 kernel requires only 16 multiplications instead of standard 36 multiplications: a 2.25x speedup! Modern GPU libraries (cuDNN) automatically dispatch Winograd for 3x3 convolutions.

## Exam Important Points

- Full derivation of Depthwise Separable Convolution computational savings ratio: (1/N) + (1/D_K^2).
- Two-step decomposition: Depthwise (spatial) + Pointwise (cross-channel).
- Convolution Theorem and FFT-based convolution principle.
- Winograd algorithm multiplication reduction for 3x3 filters.

## Worked Numerical Problems (Hand-Derived Step-by-Step)

### Problem 1 (Basic): MobileNet Depthwise Separable FLOP Calculation

Statement: A convolutional layer operates on an input feature map of size 28x28 with M = 64 input channels and produces N = 128 output channels using 3x3 filters (stride = 1, same padding). (a) Calculate the multiplication operations for a standard convolution, (b) Calculate the operations for depthwise convolution, (c) Calculate the operations for pointwise convolution, and (d) Determine the total operations and reduction percentage.

Given: D_F = 28, M = 64, N = 128, D_K = 3.

**Governing Formula: Std = D_F^2 * M * N * D_K^2; DW = D_F^2 * M * D_K^2; PW = D_F^2 * M * N; Total_Sep = DW + PW**

#### Step-by-Step Calculation:

Step 1: Compute Standard Convolution operations: Std = (28 * 28) * 64 * 128 * (3 * 3) = 784 * 64 * 128 * 9 = 57,802,752 multiplications.

Step 2: Compute Depthwise Convolution operations: DW = (28 * 28) * 64 * (3 * 3) = 784 * 64 * 9 = 451,584 multiplications.

Step 3: Compute Pointwise (1x1) Convolution operations: PW = (28 * 28) * 64 * 128 * (1 * 1) = 784 * 64 * 128 * 1 = 6,422,528 multiplications.

Step 4: Total Separable Operations = 451,584 + 6,422,528 = 6,874,112 multiplications.

Step 5: Compute reduction percentage: Ratio = 6,874,112 / 57,802,752 = 0.11892 (11.89%). Reduction = (1 - 0.11892) * 100 = 88.11% reduction!

**[ FINAL EVALUATED ANSWER: Standard = 57.8M ops. Separable = 6.87M ops (88.11% computation reduction, ~8.4x speedup). ]**

Pedagogical Note: Depthwise separable convolution reduces operations from 57.8 million to 6.87 million while retaining identical channel depth and spatial resolution.

### Problem 2 (Advanced (KTU Exam Level)): Winograd Minimal Filtering Multiplications Reduction Proof

Statement: The Winograd minimal filtering algorithm F(m, r) computes m output values using an r-tap FIR filter with m + r - 1 multiplications, whereas standard direct convolution requires m * r multiplications. For 1D convolution F(2, 3) computing 2 outputs with a 3-tap filter: (a) Compare multiplications required by direct vs Winograd convolution, (b) Extend to 2D convolution F(2x2, 3x3) which computes 2x2 outputs using a 3x3 filter, and (c) Calculate the exact theoretical speedup factor achieved by Winograd on modern GPU engines.

Given: 1D: m = 2, r = 3; 2D: F(2x2, 3x3).

**Governing Formula: 1D Direct = m * r; 1D Winograd = m + r - 1; 2D Direct = m^2 * r^2; 2D Winograd = (m + r - 1)^2**

#### Step-by-Step Calculation:

Step 1: 1D Convolution: Direct multiplications = m * r = 2 * 3 = 6 multiplications. Winograd multiplications = m + r - 1 = 2 + 3 - 1 = 4 multiplications. 1D savings = 6 / 4 = 1.5x speedup.

Step 2: 2D Convolution F(2x2, 3x3): Direct multiplications = m^2 * r^2 = (2^2) * (3^2) = 4 * 9 = 36 multiplications. Winograd multiplications = (m + r - 1)^2 = (2 + 3 - 1)^2 = 4^2 = 16 multiplications.

Step 3: Calculate 2D speedup factor: Speedup = 36 / 16 = 2.25x speedup! Multiplication reduction = ((36 - 16) / 36) * 100 = 55.56% reduction.

**[ FINAL EVALUATED ANSWER: 1D Winograd saves 33.3% (1.5x). 2D Winograd reduces multiplications from 36 to 16 (2.25x speedup, 55.56% reduction). ]**

Pedagogical Note: Winograd minimal filtering cuts 3x3 convolution multiplications by more than half, explaining why NVIDIA cuDNN utilizes Winograd by default for 3x3 kernels.

### Real-World Industry Case (Consumer Electronics & Edge Computer Vision): Smart Home Edge Camera Face Unlock

*Pipeline Workflow: Battery-powered doorbell camera detects motion -> Microcontroller runs MobileNet with depthwise separable convolutions -> Evaluates live facial biometrics in 30ms on a 1-Watt edge processor.*

***Industrial Impact & Benefit: Enables instant biometric door unlock without draining battery or sending private home video streams to the cloud.***

## Key Oral Viva Voce Checkpoints

**Q1: What is a Depthwise Separable Convolution?**

*Ans: A factorized convolution that breaks the standard operation into two steps: a depthwise spatial convolution applied independently per channel, followed by a 1x1 pointwise convolution across channels.*

**Q2: What is the computational reduction factor achieved by MobileNet depthwise separable convolution?**

*Ans: (1 / N) + (1 / D_K^2); for 3x3 filters, this yields an 8 to 9-fold computational speedup.*

**Q3: How does the Convolution Theorem enable fast convolution via FFT?**

*Ans: Convolution in the spatial domain corresponds to point-wise multiplication in the frequency domain: I * K = IFFT(FFT(I) odot FFT(K)).*

**Q4: For what kernel sizes is FFT convolution faster than direct convolution?**

*Ans: For large kernels (typically K >= 7x7 or 11x11); for small 3x3 kernels, FFT transform overhead exceeds direct computation.*

**Q5: What is the Winograd minimal filtering algorithm?**

*Ans: An algebraic algorithm that computes small convolutions with the minimum possible number of multiplications, speeding up 3x3 convolutions by 2.25x.*